TRAFFIQ 2025

AI Model Technical Report

Autonomous Vehicle Navigation using Behavioral Cloning

Developed by:

Sagnick Paul
Sagnik De
Subhrajit Mukherjee
Ritabrata Das

Organized by:

IET On-Campus JU × JUMPER

Jadavpur University

April 2026

Table of Contents

1 Executive Summary 2
2 System Overview 2
2.1 High-Level Pipeline 2
2.2 Hardware Platform 2
3 Dataset 2
3.1 Source 2
3.2 Dataset Statistics 3
3.3 Data Loading & Path Fixing 3
4 Data Preprocessing & Augmentation 3
4.1 Multi-Camera Strategy 3
4.2 Image Preprocessing Pipeline 4
5 Model Architecture 4
5.1 NVIDIA Dave-2 (End-to-End CNN) 4
5.2 Layer-by-Layer Architecture 4
5.3 Design Rationale 4
5.4 Training Configuration 4
6 Training Results 4
6.1 Loss Curves 4
6.2 Observations 5
7 CARLA Dataset Training for Real-World Implementation 5
7.1 Motivation 5
7.2 Why CARLA? 5
7.3 Data Collection Procedure 7
7.4 Fine-Tuning Strategy 7
7.5 Impact on Real-World Performance 8
8 Model Export & Deployment Format 8
8.1 PyTorch to ONNX Conversion 8
8.2 Runtime Inference on Raspberry Pi 8
9 Real-Time Drive Script (`drive.py`) 8
9.1 Camera Interface 8
9.2 Steering Angle to PWM Mapping 9
9.3 Safe-Stop Mechanism 9
10 Compliance with TRAFFIQ Rules 9
11 Limitations & Planned Improvements 9
11.1 Known Limitations 9
11.2 Planned Improvements for Final Round 10
12 Technology Stack 10
13 Obstacle Detection Module (YOLOv8n) 11
13.1 Overview 11
13.2 Dataset 11
13.3 Model: YOLOv8 Nano 11
13.4 Training Code 11
13.5 Integration with Drive Pipeline 12
13.6 Why YOLOv8n for Edge Deployment 12
13.7 Two-Model Architecture Summary 13

1. Executive Summary

This report describes our end-to-end AI system for autonomous vehicle navigation developed for the TRAFFIQ competition. Our approach is based on **Behavioral Cloning (BC)**, a supervised imitation learning technique in which a deep convolutional neural network learns to map raw camera images directly to steering commands, replicating the behaviour of a human expert driver.

The model is based on the well-established **NVIDIA Dave-2 (End-to-End)** architecture, trained initially on the **Udacity Behavioural Cloning dataset** and subsequently fine-tuned on a **CARLA simulator dataset** for improved real-world generalisation. The model is exported to ONNX format for efficient real-time inference on the competition-provided Raspberry Pi 4B 8 GB hardware. The complete pipeline from camera capture to motor control runs locally on the Pi with no external communication.

2. System Overview

2.1 High-Level Pipeline

The autonomous driving pipeline consists of five sequential stages:

1. **Camera Frame Capture** — Raspberry Pi Camera V2 streams 30 FPS video.
2. **Preprocessing** — Frames are converted to YUV colour space, cropped, and resized to match the training distribution exactly.
3. **Model Inference** — The preprocessed frame is fed into the ONNX model running on ONNX Runtime (ARM-optimised). The model outputs a single steering angle value in the range $[-1, +1]$.
4. **Control Mapping** — The predicted steering angle is mapped to a PWM pulse width sent to the steering servo.
5. **Actuation** — A fixed-throttle PWM signal drives the ESC/motor; steering PWM drives the servo.

2.2 Hardware Platform

Table 1: Hardware components and their roles

Component	Specification	Role
Raspberry Pi 4B	8 GB RAM, ARM Cortex-A72	Main compute unit — model inference + control
RPi Camera V2	Sony IMX219, 8 MP, 30 FPS	Real-time visual input
Steering Servo	Standard PWM servo (1000–2000 µs)	Lane-tracking steering control
ESC + DC Motor	Standard RC car ESC	Throttle / forward motion
GPIO (pigpio)	Hardware PWM, GPIO 18 & 19	Precise pulse-width modulation

3. Dataset

3.1 Source

We used the **Udacity Self-Driving Car Behavioural Cloning Dataset** (Jungle Track variant), publicly available on Kaggle. The dataset was recorded from a simulated driving

environment using the Udacity Car Simulator.

3.2 Dataset Statistics

Table 2: Dataset summary statistics

Property	Value
Total Samples (rows)	3,404
Training Set (80%)	2,723 samples
Validation Set (10%)	340 samples
Test Set (10%)	341 samples
Input Columns	center, left, right camera image paths
Target Column	steering angle (float, range $\approx [-1, +1]$)
Additional Columns	throttle, brake, speed
Image Resolution (raw)	320×160 pixels, RGB

3.3 Data Loading & Path Fixing

The original CSV was recorded on a Windows machine with backslash-separated paths. A path-fixing function was applied to normalise all image paths to the Linux filesystem:

```
def path_leaf(path):
    head, tail = ntpath.split(path)
    return tail

data['center'] = data['center'].apply(path_leaf)
data['left']   = data['left'].apply(path_leaf)
data['right']  = data['right'].apply(path_leaf)
data.head()
```

Listing 1: Path normalisation for Linux filesystem

After fixing, 100% of image paths were validated as accessible on disk.

4. Data Preprocessing & Augmentation

4.1 Multi-Camera Strategy

The Udacity simulator records three simultaneous camera feeds: centre, left, and right. During training, a camera is randomly selected for each sample. A steering correction of $+0.2$ is applied for left-camera images (car should steer right to re-centre) and -0.2 for right-camera images. This effectively triples the usable dataset size and teaches the model recovery behaviour.

Table 3: Multi-camera steering corrections

Camera	Steering Correction	Purpose
Centre	0.0 (none)	Primary driving line
Left	$+0.2$	Teaches rightward correction
Right	-0.2	Teaches leftward correction

4.2 Image Preprocessing Pipeline

The image preprocessing pipeline transforms raw camera images into a format suitable for the model. Images are converted from BGR to RGB format and cropped to remove irrelevant regions such as the sky and vehicle hood, focusing only on the road. The cropped images are resized to a fixed resolution of 66×200 pixels to match the model input.

Pixel values are normalized to the range [0, 1] to improve training stability. Additionally, data augmentation techniques such as random brightness adjustment, shadow addition, flipping, and translation are applied to increase dataset diversity and improve robustness under varying lighting and driving conditions.

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = cv2.cvtColor(img, cv2.COLOR_RGB2YUV)
img = img[60:-25, :, :]          # crop sky and hood
img = cv2.resize(img, (200, 66))
```

Listing 2: Core image preprocessing code

5. Model Architecture

5.1 NVIDIA Dave-2 (End-to-End CNN)

The model architecture is based on the end-to-end learning approach proposed by NVIDIA for self-driving cars [1]. It consists of a series of convolutional layers followed by fully connected layers. The convolutional layers automatically extract spatial features such as lane markings and road edges, while the fully connected layers map these features to a steering angle output.

The model takes a 3-channel image as input and produces a single continuous value representing the steering command. Activation functions such as ReLU introduce non-linearity, enabling the model to learn complex driving patterns.

5.2 Layer-by-Layer Architecture

5.3 Design Rationale

- The system performs **end-to-end learning**, directly mapping input images to steering angles without explicit feature engineering.
- **Data balancing and augmentation** address biases such as over-representation of straight driving and improve performance in diverse conditions including low light.
- **Transfer learning and fine-tuning** extend the model's capability to handle new scenarios such as obstacle avoidance while preserving lane-following behaviour.
- The overall design prioritises **efficiency**, making it suitable for deployment on resource-constrained devices like the Raspberry Pi.

5.4 Training Configuration

6. Training Results

6.1 Loss Curves

The model was trained for 20 epochs. Both training loss (MSE) and validation loss converged steadily, with no significant overfitting observed.

Table 4: NVIDIA Dave-2 layer-by-layer architecture

Layer	Type	Configuration	Output Shape
Input	Input	Shape $(66, 200, 3)$	(None, 66, 200, 3)
L1	Lambda (Norm.)	$x \div 255.0$	(None, 66, 200, 3)
L2	Conv2D	24 filters, 5×5, stride 2×2, ReLU	(None, 31, 98, 24)
L3	Conv2D	36 filters, 5×5, stride 2×2, ReLU	(None, 14, 47, 36)
L4	Conv2D	48 filters, 5×5, stride 2×2, ReLU	(None, 5, 22, 48)
L5	Conv2D	64 filters, 3×3, stride 1×1, ReLU	(None, 3, 20, 64)
L6	Conv2D	64 filters, 3×3, stride 1×1, ReLU	(None, 1, 18, 64)
L7	Flatten	—	(None, 1152)
L8	Dense	100 units, ReLU	(None, 100)
L9	Dense	50 units, ReLU	(None, 50)
L10	Dense	10 units, ReLU	(None, 10)
L11	Dense (Output)	1 unit, Linear	(None, 1)

6.2 Observations

- The best validation loss of 0.0809 MSE was achieved at Epoch 20, with validation MAE of 0.2132 (approximately 12.7° on a $[-1, +1]$ normalised scale).
- Training and validation curves track closely, indicating the model has not overfit to the training data.
- The multi-camera augmentation strategy with steering correction effectively regularised the model and improved recovery behaviour.
- MAE of ≈ 0.21 corresponds to an average steering error of roughly 4.7° in real-world terms — sufficient for smooth lane-following at moderate speeds.

7. CARLA Dataset Training for Real-World Implementation

7.1 Motivation

While the Udacity dataset provided a solid foundation for learning basic lane-following behaviour, it was recorded in a relatively simple simulated environment with limited variation in road textures, lighting conditions, and urban complexity. To improve generalisation towards real-world deployment, we subsequently trained the model on data collected from the **CARLA simulator** [3] — a high-fidelity, open-source autonomous driving simulator designed to closely replicate real-world urban driving scenarios.

7.2 Why CARLA?

CARLA provides a significantly more photorealistic and diverse environment compared to the Udacity simulator:

Table 5: Hyperparameters and training settings

Hyperparameter	**Value**	**Justification**
Batch Size	64	Balances training stability and efficiency.
Learning Rate (initial)	10^{-4}	Stable learning without large loss oscillations.
Learning Rate (fine-tuning)	10^{-5}	Prevents drastic weight updates during fine-tuning.
Optimizer	Adam	Adaptive learning rates for deep networks.
Loss Function	Weighted MSE	Emphasises larger steering angles for sharp turns.
Number of Epochs	40–50	Sufficient training while avoiding overfitting.
Input Image Size	$(3, 66, 200)$	Matches NVIDIA architecture; reduces compute.
Data Augmentation	Brightness, Shadow, Flip, Translate	Improves generalisation under varying conditions.
Steering Correction (L/R)	± 0.3	Teaches recovery when vehicle deviates from centre.
Dropout	Not used	Regularisation handled via augmentation.
Activation Function	ReLU	Introduces non-linearity; avoids vanishing gradients.
Output Activation	Tanh	Restricts steering output to $[-1, 1]$.
Dataset Balancing	Reduced straight, boosted turns	Prevents straight-driving bias; improves turning.

Table 6: Training and validation metrics per epoch

Epoch	**Train Loss (MSE)**	**Val Loss (MSE)**	**Train MAE**	**Val MAE**
1	0.1832	0.1241	0.3218	0.2640
5	0.1202	0.1098	0.2655	0.2529
10	0.0998	0.0970	0.2409	0.2285
15	0.0844	0.0898	0.2246	0.2254
16	0.0835	0.0830	0.2189	0.2157
20	0.0745	0.0809	0.2071	0.2132

- **Photorealistic rendering:** CARLA uses Unreal Engine 4, producing images with realistic road markings, traffic signs, buildings, and dynamic lighting that closely approximate real camera footage.
- **Diverse weather and lighting:** Training data was collected across multiple weather

presets (clear, cloudy, rainy, wet roads) and times of day (midday, sunset, night), exposing the model to the full range of conditions it may encounter.

- **Multiple map environments:** CARLA's built-in town maps (Town01–Town07) include a variety of road layouts, intersections, and lane configurations, preventing overfitting to a single track topology.
- **Sensor parity:** CARLA's virtual RGB camera sensor can be calibrated to match the field of view and resolution of the Raspberry Pi Camera V2, reducing the sim-to-real gap in image statistics.

7.3 Data Collection Procedure

Training data was collected by running an autopilot agent in CARLA and recording synchronised camera frames alongside ground-truth steering angles:

```python
import carla, cv2, csv

client  = carla.Client('localhost', 2000)
world   = client.load_world('Town03')
vehicle = world.spawn_actor(bp, spawn_point)
vehicle.set_autopilot(True)

camera_bp = world.get_blueprint_library().find(
    'sensor.camera.rgb')
camera_bp.set_attribute('image_size_x', '320')
camera_bp.set_attribute('image_size_y', '160')

def save_frame(image, control):
    array = np.frombuffer(image.raw_data,
                          dtype=np.uint8)
    array = array.reshape((160, 320, 4))[:, :, :3]
    cv2.imwrite(f'frames/{image.frame}.png', array)
    writer.writerow([f'{image.frame}.png',
                     control.steer])
```

Listing 3: CARLA data collection snippet

Table 7: CARLA dataset collection settings

Property	Value
Simulator	CARLA 0.9.x
Maps Used	Town01, Town02, Town03, Town05
Weather Presets	ClearNoon, CloudySunset, WetNoon, HardRainNight
Driving Agent	CARLA built-in autopilot
Camera Resolution	320×160 pixels, RGB
Target Column	Steering angle (float, $[-1, +1]$)

7.4 Fine-Tuning Strategy

Rather than training from scratch on the CARLA dataset, we adopted a **transfer learning** approach: the weights from the Udacity-trained model were used as initialisation, and the model was fine-tuned on the CARLA data with a reduced learning rate of 10^{-5}.

This preserved the low-level visual feature representations learned from the Udacity data while adapting the higher-level decision layers to the richer CARLA environment.

- Convolutional layers (L2–L6): frozen for the first 10 epochs to stabilise feature extraction.
- Fully connected layers (L8–L11): unfrozen throughout, allowing the decision mapping to adapt to the new data distribution.
- After 10 epochs, all layers were unfrozen for end-to-end fine-tuning at learning rate 10^{-5}.

7.5 Impact on Real-World Performance

Training on the CARLA dataset produced several measurable improvements in generalisation:

- **Lighting robustness:** The model became significantly more stable under low-light and shadowed conditions, attributable to the diversity of CARLA weather presets in the training data.
- **Lane recovery:** Exposure to varied road textures and lane marking styles improved recovery behaviour when the vehicle drifted towards lane boundaries.
- **Reduced sim-to-real gap:** CARLA's photorealistic rendering resulted in a narrower distribution shift between training images and real-world camera frames compared to the Udacity simulator.

8. Model Export & Deployment Format

8.1 PyTorch to ONNX Conversion

The trained model was converted from PyTorch to ONNX format to enable deployment on the Raspberry Pi. ONNX provides a framework-independent representation for efficient inference. The conversion was performed using `torch.onnx.export()` with a dummy input of shape $(1, 3, 66, 200)$, matching the model's input. The model was set to evaluation mode to ensure consistent behaviour.

8.2 Runtime Inference on Raspberry Pi

Inference is performed using `onnxruntime`:

```python
import onnxruntime as ort

session    = ort.InferenceSession('self_driving_model.onnx')
input_name = session.get_inputs()[0].name
steering   = session.run(None, {input_name: frame})[0][0][0]
```

Listing 4: ONNX Runtime inference on Raspberry Pi

Expected inference latency on RPi 4B: approximately 50–100 ms per frame, yielding 10–20 FPS — adequate for the arena speeds expected.

9. Real-Time Drive Script (`drive.py`)

9.1 Camera Interface

The Raspberry Pi Camera V2 is accessed via OpenCV's `VideoCapture` interface at `/dev/video0`. Frames are captured at 320×240 resolution before being downscaled

by the preprocessing pipeline.

9.2 Steering Angle to PWM Mapping

The model output (range ≈ -1.0 to $+1.0$) is mapped to a servo PWM pulse width:

```
pwm = int(1500 + steering_angle * 500)
pwm = clip(pwm, 1000, 2000)   # microseconds
```

Listing 5: Steering angle to PWM conversion

Table 8: Steering angle to PWM mapping

Steering Angle	PWM (µs)	Physical Meaning
−1.0 (full left)	1000	Maximum left lock
0.0 (straight)	1500	Centre / straight ahead
+1.0 (full right)	2000	Maximum right lock

9.3 Safe-Stop Mechanism

As required by the TRAFFIQ rules, the system implements a safe-stop function that is triggered on camera failure, `KeyboardInterrupt`, or unexpected exceptions. On activation it immediately sets throttle to neutral (1500 µs) and steering to centre (1500 µs) before releasing GPIO resources.

10. Compliance with TRAFFIQ Rules

Table 9: TRAFFIQ compliance checklist

Rule	Our Implementation
Fully autonomous once started	Run loop executes without human input after launch.
No hardcoded coordinates/path scripts	Navigation is 100% based on real-time camera inference.
No external communication during run	All computation runs locally on RPi; Wi-Fi/BT disabled.
Safe-stop mechanism	Implemented in `finally` block; triggers on any failure.
Approved hardware only	Computation runs entirely on provided RPi 4B.
Robust to lighting changes	YUV colour space separates luminance; model trained on varied conditions including CARLA night/rain presets.
Own model / credited open source	NVIDIA Dave-2 architecture (cited); Udacity + CARLA datasets (cited); all code original.
Model inference within latency threshold	ONNX Runtime ARM-optimised; ∼50–100 ms per frame.

11. Limitations & Planned Improvements

11.1 Known Limitations

- **Residual Sim-to-Real Gap:** Although CARLA training substantially reduced the distribution shift, a gap between simulated and real-world images persists. Domain randomisation or real-world data collection would further close this gap.

- **Fixed Throttle:** Speed is currently constant. Dynamic speed control based on curvature or obstacle proximity is not yet implemented.
- **No Obstacle Avoidance:** The current model predicts only steering; no object detection module is integrated for obstacle avoidance.
- **Crop Sensitivity:** The hardcoded crop `[60:-25]` assumes the training camera height. Adjustments may be required based on actual mounting position.

11.2 Planned Improvements for Final Round

- **Extended CARLA testing:** Full closed-loop evaluation of the behavioral cloning model within CARLA — using the simulator's Python API to drive and score the model across multiple towns and weather conditions before physical deployment. This will serve as a standardised benchmark for iterative model improvement.
- **CARLA scenario testing:** Evaluation on CARLA's built-in scenario runner (`ScenarioRunner`) for structured tests including lane changes, junction navigation, and pedestrian avoidance.
- Camera calibration and histogram equalisation to bridge the remaining sim-to-real colour gap.
- Adaptive throttle control: reduce speed proportionally when $|\text{steering_angle}| > 0.3$.
- Integration of a lightweight YOLOv5n or MobileNet-SSD object detector for obstacle detection as a second inference pass.
- Real-time frame capture using the `picamera2` library for lower latency on Raspberry Pi OS Bookworm.

12. Technology Stack

Table 10: Full technology stack

Component	Technology	Version
Deep Learning Framework	PyTorch	2.x
Model Architecture	NVIDIA Dave-2 (Custom Keras Implementation)	—
Primary Training Dataset	Udacity Behavioural Cloning (Jungle Track)	—
Real-World Training Dataset	CARLA Simulator (Towns 01–05)	0.9.x
Model Export	`torch.onnx.export()`	1.17.0
Inference Runtime	ONNX Runtime (ARM)	Latest
Computer Vision	OpenCV (`cv2`)	4.x
GPIO Control	pigpio	Latest
Data Processing	NumPy, Pandas	Latest
Visualisation	Matplotlib, Seaborn	Latest
Edge Hardware	Raspberry Pi 4B 8 GB	—
Camera	Raspberry Pi Camera Module V2	—

13. Obstacle Detection Module (YOLOv8n)

13.1 Overview

In addition to the core behavioral cloning steering model, we have integrated a real-time obstacle detection module using **YOLOv8 Nano (YOLOv8n)** — the smallest and fastest variant in the YOLOv8 family by Ultralytics [7]. This module processes the same camera feed in parallel and triggers evasive or braking behaviour when an obstacle is detected within a defined proximity threshold.

13.2 Dataset

The obstacle detection model was trained on the **Udacity Self-Driving Car Obstacles Dataset** [8], downloaded from Kaggle. The dataset is structured in YOLO format:

Table 11: Obstacle detection dataset details

Component	Details
Dataset Source	Kaggle: `barkataliarbab/udacity-self-driving-car-obstacles-dataset`
Format	YOLO format (`class cx cy w h`, normalised coordinates)
Directory Structure	`export/images/` and `export/labels/`
Config File	`data.yaml` (class names, train/val/test paths)
Label Format	`1 0.104 0.55 0.208 0.273` (class x_c y_c w h)

13.3 Model: YOLOv8 Nano

YOLOv8n was selected for its inference speed on Raspberry Pi 4B and the relatively simple detection task (detecting obstacles in a bounded arena).

Table 12: YOLOv8n model configuration

Property	Value
Model	YOLOv8n (Nano)
Framework	Ultralytics 8.4.34
Base Weights	`yolov8n.pt` (COCO pretrained)
Task	Object Detection (single class: obstacle)
Input Size	640×640 pixels
Training Epochs	50
Batch Size	16 (default)
Augmentation	Default Ultralytics augmentation pipeline
Backend	PyTorch 2.1.0 + CPU (Intel Xeon, Colab training)

13.4 Training Code

The model was trained using the Ultralytics high-level training API:

```python
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
model.train(
    data=os.path.join(path, 'data.yaml'),
    epochs=50,
```

```
    imgsz=640
)
```

Listing 6: YOLOv8n training code

13.5 Integration with Drive Pipeline

On the Raspberry Pi, YOLOv8n inference runs as a second pass on the same captured frame, immediately after the steering model inference.

Table 13: Obstacle detection response logic

Condition	Action
No obstacle (confidence < threshold)	Normal steering from behavioral cloning model
Obstacle detected, bounding box small (far)	Slight steering correction away from obstacle centre
Obstacle detected, bounding box large (close)	Reduce throttle; increase steering correction
Obstacle fills >30% of frame width	Emergency stop (safe-stop triggered)

```python
from ultralytics import YOLO

yolo    = YOLO('yolov8n_obstacles.pt')
results = yolo(frame, verbose=False)
boxes   = results[0].boxes

if len(boxes) > 0:
    x1, y1, x2, y2 = boxes[0].xyxy[0]
    box_width = (x2 - x1) / frame.shape[1]
    if box_width > 0.30:
        safe_stop()     # obstacle too close
```

Listing 7: Obstacle detection integration on Raspberry Pi

13.6 Why YOLOv8n for Edge Deployment

- **Model Size:** YOLOv8n weights are approximately 6.2 MB — small enough to load quickly on the Pi.
- **Inference Speed:** On RPi 4B (ARM Cortex-A72), YOLOv8n achieves approximately 200–400 ms per frame in CPU-only mode. Combined with the 50–100 ms ONNX steering inference, total loop latency is ∼300–500 ms, giving ∼2–3 FPS obstacle awareness — sufficient for the arena's low speed.
- **COCO Pretraining:** The `yolov8n.pt` base weights include general object classes, providing good initialisation before fine-tuning.
- **Ultralytics API:** The unified train/infer API reduces boilerplate and ensures reliable data loading from YOLO-format datasets.

Table 14: Dual-model deployment summary

Model	Architecture	Task	Output	Format
Steering Model	NVIDIA Dave-2 CNN	Predict steering angle from image	Float $[-1, +1]$	ONNX (`.onnx`)
Obstacle Detector	YOLOv8n	Detect obstacles in frame	Bboxes + confidence	PyTorch (`.pt`)

13.7 Two-Model Architecture Summary

Both models run sequentially on the same camera frame. The obstacle detector acts as a **safety override layer** on top of the behavioral cloning steering output, ensuring the vehicle can react to unexpected objects in the arena without relying on the steering model alone.

13. References

[1] Bojarski, M. et al. (2016). *End to End Learning for Self-Driving Cars*. NVIDIA Technical Report. arXiv:1604.07316.

[2] Udacity Self-Driving Car Behavioural Cloning Dataset (Jungle Track). Available on Kaggle: https://www.kaggle.com/datasets/andy8744/udacity-self-driving-car-behavioural-cloning

[3] Dosovitskiy, A. et al. (2017). *CARLA: An Open Urban Driving Simulator*. Conference on Robot Learning (CoRL). https://carla.org/

[4] ONNX Runtime Documentation. https://onnxruntime.ai/

[5] pigpio Library Documentation. http://abyz.me.uk/rpi/pigpio/

[6] tf2onnx: Convert TensorFlow models to ONNX. https://github.com/onnx/tensorflow-onnx

[7] Jocher, G. et al. (2023). *Ultralytics YOLOv8*. https://github.com/ultralytics/ultralytics

[8] Udacity Self-Driving Car Obstacles Dataset. Kaggle: https://www.kaggle.com/datasets/barkataliarbab/udacity-self-driving-car-obstacles-dataset